Паттерны проектирования
Часть 2

Грачев Д.Г.

ПОСРЕДНИК

Посредник

Объекты общаются через посредника, а не напрямую друг с другом.

```
// Обратите внимание на сущности!
public class Main {
    public static void main(String[] args) {
        ChatMediator chat = new ChatRoom();

        User user1 = new ChatUser(chat, "Анна");
        User user2 = new ChatUser(chat, "Иван");
        User user3 = new ChatUser(chat, "Мария");

        chat.addUser(user1);
        chat.addUser(user2);
        chat.addUser(user3);

        user1.send("Всем привет!");
    }
}
```

Посредник

```java
// Конкретный пользователь
class ChatUser extends User {
    public ChatUser(ChatMediator mediator, String name) {
        super(mediator, name);
    }

    public void send(String message) {
        System.out.println(name + " отправляет: " + message);
        mediator.sendMessage(message, this);
    }

    public void receive(String message) {
        System.out.println(name + " получил: " + message);
    }
}
```

Как же тогда должен выглядеть сам User?

Посредник

```java
// Пользователь
abstract class User {
    protected ChatMediator mediator;
    protected String name;

    public User(ChatMediator mediator, String name) {
        this.mediator = mediator;
        this.name = name;
    }

    public abstract void send(String message);
    public abstract void receive(String message);
}
```

Почему?

Посредник

```
interface ChatMediator {
    void sendMessage(String message, User user);
    void addUser(User user);
}
```

Кто же должен тогда реализовывать addUser и sendMessage?

Посредник

```
interface ChatMediator {
    void sendMessage(String message,
                     User user);
    void addUser(User user);
}
```

```
class ChatRoom implements ChatMediator {
    private List<User> users = new ArrayList<>();

    public void addUser(User user) {
        users.add(user);
    }

    public void sendMessage(String message, User
sender) {
        for (User user : users) {
            // Не отправляем сообщение отправителю
            if (user != sender) {
                user.receive(message);
            }
        }
    }
}
```

За реализацию отвечает конкретная реализация медиатора. В системах их может быть множество, а интерфейс один

ХРАНИТЕЛЬ

ХРАНИТЕЛЬ

```
public static void main(String[] args) {
    TextEditor editor = new TextEditor();
    History history = new History();

    editor.write("Привет, ");
    history.push(editor.save());

    editor.write("мир!");
    System.out.println("Текст: " + editor.getText());  // Привет, мир!

    // Отмена
    editor.restore(history.pop());
    System.out.println("После отмены: " + editor.getText());  // Привет,
}
```

```java
class TextEditor {
    private String text = "";

    public void write(String words) {
        text += words;
    }

    public String getText() {
        return text;
    }

    public Memento save() {
        return new Memento(text);
    }

    public void restore(Memento memento) {
        text = memento.getText();
    }

    static class Memento {
        private String text;

        private Memento(String text) {
            this.text = text;
        }

        private String getText() {
            return text;
        }
    }
}
```

```java
class History {
    private Stack<TextEditor.Memento> history = new Stack<>();

    public void push(TextEditor.Memento memento) {
        history.push(memento);
    }

    public TextEditor.Memento pop() {
        return history.pop();
    }
}
```

Наблюдатель

Наблюдатель

```
// Интерфейс наблюдателя
interface Observer {
    void update(String news);
}

// Интерфейс субъекта
interface Subject {
    void attach(Observer observer);
    void detach(Observer observer);
    void notifyObservers();
}
```

Основное: Сам наблюдатель обрабатывает события, в то время как субъект управляет подпиской.

Наблюдатель

```java
public class Main {
    public static void main(String[] args) {
        NewsAgency agency = new NewsAgency();

        Observer user1 = new EmailSubscriber("Анна");
        Observer user2 = new SMSSubscriber("Иван");

        agency.attach(user1);
        agency.attach(user2);

        agency.publishNews("Java 21 вышла!");
    }
}
```

```java
class NewsAgency implements Subject {
   private List<Observer> observers = new
ArrayList<>();
   private String latestNews;

   public void attach(Observer observer) {
       observers.add(observer);
   }

   public void detach(Observer observer) {
       observers.remove(observer);
   }

   public void notifyObservers() {
       for (Observer observer : observers) {
           observer.update( latestNews);
       }
   }

   public void publishNews(String news) {
       this.latestNews = news;
       notifyObservers();
   }
}
```

Именно новостное агентство управляет, к примеру, платными подписками и обязуется предоставить самые актуальные новости подписантам, те наблюдателям.

```java
// Конкретные наблюдатели
class EmailSubscriber implements Observer
{
    private String name;

    public EmailSubscriber(String name) {
        this.name = name;
    }

    public void update(String news) {
        System.out.println(name + "
получил новость по email: " + news);
    }
}
```

А вот сами наблюдатели могут быть различными: кто-то получает данные через почту или смс

```java
class SMSSubscriber implements Observer {
    private String name;

    public SMSSubscriber(String name) {
        this.name = name;
    }

    public void update(String news) {
        System.out.println(name + " получил SMS: "
+ news);
    }
}
```

Самое важное, что для агентства они являются именно наблюдателями имеющими одно важное свойство - update!

Состояние

Состояние (state)

Объект меняет поведение в зависимости от своего состояния.

```
// Использование
public class Main {
    public static void main(String[] args) {
        Order order = new Order();

        order.process(); // Заказ создан

        order.setState(new PaidState());
        order.process(); // Заказ оплачен

        order.setState(new ShippedState());
        order.process(); // Заказ отправлен

        order.setState(new DeliveredState());
        order.process(); // Заказ доставлен
    }
  }
```

Не стоит путать с паттерном state machine! Главная особенность: в паттерне state в зависимости от состояния меняется само поведение объекта, в то время как в statemachine есть описание конечного автомата, где описываются переходы между состояниями в зависимости от событий!

Состояние (state)

```
class Order {
    private State state;

    public Order() {
        state = new NewOrderState();
    }

    public void setState(State state)
{
        this.state = state;
    }

    public void process() {
        state.handle();
    }
}
```

Как видите поведение заказа физически зависит от того состояние в котором оно находится, при этом интерфейс един между состояниями, что позволяет добавлять новые без изменения существующих!

```java
class NewOrderState implements State {
    public void handle() {
        System.out.println("Заказ создан. Ожидает оплаты." );
    }
}

class PaidState implements State {
    public void handle() {
        System.out.println("Заказ оплачен. Готовится к отправке." );
    }
}

class ShippedState implements State {
    public void handle() {
        System.out.println("Заказ отправлен. Ожидает получения." );
    }
}

class DeliveredState implements State {
    public void handle() {
        System.out.println("Заказ доставлен. Спасибо за покупку!" );
    }
}
```

Стратегия

Стратегия

```java
interface PaymentStrategy {
    void pay(int amount);
}

class CreditCardPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Оплачено " + amount + " руб. кредитной картой");
    }
}

class PayPalPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Оплачено " + amount + " руб. через PayPal");
    }
}

class CashPayment implements PaymentStrategy {
    public void pay(int amount) {
        System.out.println("Оплачено " + amount + " руб. наличными");
    }
  }
```

Можно подменять алгоритм на лету, не меняя код класса.

Стратегия

```java
// Контекст
class ShoppingCart {
    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(PaymentStrategy strategy) {
        this.paymentStrategy = strategy;
    }

    public void checkout(int amount) {
        if (paymentStrategy == null) {
            System.out.println("Выберите способ оплаты!");
        } else {
            paymentStrategy.pay(amount);
        }
    }
}
```

Физически можно подменять способы оплаты на лету. К примеру когда у пользователя несколько карт или способов оплаты и вам надо хоть откуда-то списать деньги =-)

Стратегия

```java
public class Main {
    public static void main(String[] args) {
        ShoppingCart cart = new ShoppingCart();

        cart.checkout(1000); // Способ не выбран

        cart.setPaymentStrategy(new CreditCardPayment());
        cart.checkout(1000); // Оплата картой

        cart.setPaymentStrategy(new PayPalPayment());
        cart.checkout(500); // Оплата PayPal
    }
  }
```

Самое важное, мы можем добавлять способы оплаты и не усложнять более высокоуровневую логику . Другими словами - никаких бесконечных if-else

Шаблонный метод

Шаблонный метод

```java
// Абстрактный класс с шаблонным методом
abstract class BeverageMaker {
    // Шаблонный метод
    public final void makeBeverage() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    private void boilWater() {
        System.out.println("Кипятим воду");
    }

    private void pourInCup() {
        System.out.println("Наливаем в чашку");
    }

    // Шаги, которые реализуют подклассы
    protected abstract void brew();
    protected abstract void addCondiments();
  }
```

Общий алгоритм определен в базовом классе, а конкретные шаги реализуют наследники.

Шаблонный метод

```java
class TeaMaker extends BeverageMaker {
    protected void brew() {
        System.out.println("Завариваем чай");
    }

    protected void addCondiments() {
        System.out.println("Добавляем лимон");
    }
}

class CoffeeMaker extends BeverageMaker {
    protected void brew() {
        System.out.println("Завариваем кофе");
    }

    protected void addCondiments() {
        System.out.println("Добавляем сахар и молоко");
    }
  }
```

```java
public class Main {
    public static void main(String[] args) {
        System.out.println("Делаем чай:");
        TeaMaker tea = new TeaMaker();
        tea.makeBeverage();

        System.out.println("\nДелаем кофе:");
        CoffeeMaker coffee = new CoffeeMaker();
        coffee.makeBeverage();
    }
  }
```

Посетитель (Visitor)

Посетитель (Visitor)

```java
public class Main {
    public static void main(String[] args) {
        Shape[] shapes = {new Circle(), new Rectangle()};

        Visitor areaCalculator = new AreaCalculator();
        Visitor drawer = new Drawer();

        System.out.println("Вычисляем площади:");
        for (Shape shape : shapes) {
            shape.accept(areaCalculator);
        }

        System.out.println("\nРисуем фигуры:");
        for (Shape shape : shapes) {
            shape.accept(drawer);
        }
    }
}
```

Классы не меняем, но можем добавлять новые опеации!

```
Вычисляем площади:
Площадь круга: 78.53981633974483
Площадь прямоугольника: 200

Рисуем фигуры:
Рисуем круг радиусом 5
Рисуем прямоугольник 10x20
```

```java
// Интерфейс элемента, который можно посетить
interface Shape {
    void accept(Visitor visitor);
}

// Конкретные элементы
class Circle implements Shape {
    private int radius = 5;

    public int getRadius() { return radius; }

    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
}

class Rectangle implements Shape {
    private int width = 10;
    private int height = 20;

    public int getWidth() { return width; }
    public int getHeight() { return height; }

    public void accept(Visitor visitor) {
        visitor.visit(this);
    }
  }
```

Создаем конкретные объекты что могут поддерживать данный паттерн. Но для каждой операции нам нужно определить самого посетителя!

```java
// Интерфейс посетителя
interface Visitor {
    void visit(Circle circle);
    void visit(Rectangle rectangle);
}

// Конкретный посетитель - вычисление площади
class AreaCalculator implements Visitor {
    public void visit(Circle circle) {
        double area = Math.PI * circle.getRadius() * circle.getRadius();
        System.out.println("Площадь круга: " + area);
    }

    public void visit(Rectangle rectangle) {
        int area = rectangle.getWidth() * rectangle.getHeight();
        System.out.println("Площадь прямоугольника: " + area);
    }
}

// Другой посетитель - отрисовка
class Drawer implements Visitor {
    public void visit(Circle circle) {
        System.out.println("Рисуем круг радиусом " + circle.getRadius());
    }

    public void visit(Rectangle rectangle) {
        System.out.println("Рисуем прямоугольник " + rectangle.getWidth() + "x" + rectangle.getHeight());
    }
  }
```

Подписывайтесь

Ресурсы

- https://ru.wikipedia.org/wiki/%D0%A8%D0%B0%D0%B1%D0%BB%D0%BE%D0%BD_%D0%BF%D1%80%D0%BE%D0%B5%D0%BA%D1%82%D0%B8%D1%80%D0%BE%D0%B2%D0%B0%D0%BD%D0%B8%D1%8F - вики
- https://www.ozon.ru/product/head-first-patterny-proektirovaniya-2-e-izdanie-frimen-erik-robson-elizabet-380155226/?at=z6tOq2VpqcN4G0PJfPEOXZ1C2gv3VGczooVg5uyWvlBz - **Паттерны проектирования. 2-е издание | Фримен Эрик, Робсон Элизабет**
- **https://javarush.com/groups/posts/3421-shablon-nabljudateljh-observer - Наблюдатель**
- **https://habr.com/ru/articles/931962/ - state vs statemachine**